

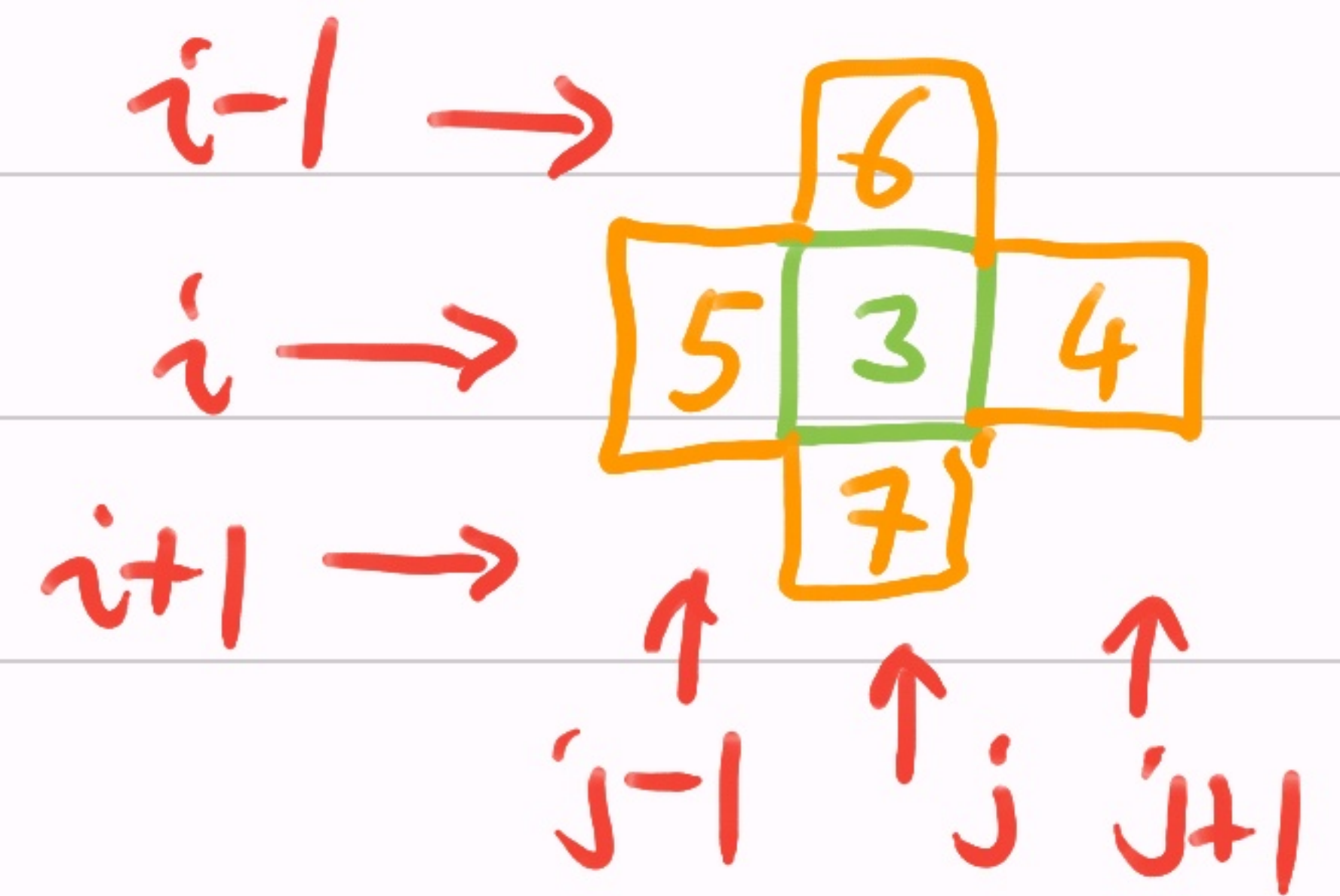
Problem: Local-Minima in a Grid

WAP : Input: $G[n][n]$ storing $n \times n$ distinct integers.

Output: Find a local-minima, i.e. an entry $G[i][j]$ smaller than its four neighbors.

▷ A local-minima exists in G .

Pf: $\because$ global-minima is also local-minima. $\square$



Ideas? (i) Single-scan of G yields a local-minima. Time $\approx O(n^2)$.

Qri: Can you reduce it? $O(n)$?

(ii) Local exploration / iteration: Go to a smaller neighbor, and repeat!
↳ if you can't, then output the current cell.

Local Explore ($G[][]$):

- $c := G[0][0];$
- while (c is NOT local-minima) $c \leftarrow$ least neighbor;
- return $c;$

▷ In this algo, no cell is visited twice.

Pf: Value c decreases in every step. $\square$
 $\Rightarrow$ while-loop halts.

Exercise: I input G on which time $\approx O(n^2)$,
 $\hookrightarrow$ so, = brute-force.

(iii) Let's make this idea more clever, by looking at a 1-D array $A[0 \dots n-1]$.

Idea: Locally explore using
binary-search & mid = $\frac{L+R}{2}$
value.



- If $A[\text{mid}]$ is not local-minima in $A[L \dots R]$, then move to a neighboring smaller element: $\text{mid}-1$ or $\text{mid}+1$; move to that half.

Lemma 1:

$$|L_{\text{new}} - R_{\text{new}}| \leq \frac{1}{2} \cdot |L - R|$$

Pf: Since we consider the mid each time. $\square$



Say, $\uparrow$ is the smallest of the three

- The process ensures the following invariant:

Left-half is used next;
 $R \leftarrow \text{mid}-1$

Lemma 2: $A[L-1] > A[L]$; $A[R] < A[R+1]$.

Pf: • If $L-1$ or $R+1$ don't exist, then it is vacuously true.

• When they exist, the pseudocode iterates following these inequalities.

• So, we get a proof by induction on the # iterations $\square$

Theorem: This algo. finds local-minima ($A[0 \dots n-1]$) in $O(\lg n)$ steps.

Pf:

- By Lemma 1, there are $\lg n$ - steps.
- At the last step: $L = R$, which means $A[L \dots R]$ is a single element x .
- By Lemma 2, $A[L-1] > x < A[R+1]$
 $\Rightarrow x$ is a local-minima. $\square$

Local-min-Array ($A[0 \dots n-1]$):

- $L = 0$; $R = n-1$;
found = False;

- while (not found) {
 - $mid \leftarrow (L+R)/2$;
 - if ($A[mid]$ is local-min) found $\leftarrow$ True;

- else if ($A[mid-1] < A[mid]$) $R \leftarrow mid-1;$
else $L \leftarrow mid+1;$ }
- return $A[mid];$

Qn: Can we extend this idea from 1-D to 2-D arrays?

- Instead of a cell in an array, think of a column in the grid.
- Consider $G[*][L \dots R]$; $mid = \frac{L+R}{2}$.
the sub-matrix.

- Find the min $G[*][mid] \Rightarrow O(n)$ -time.

$$:= \min_{0 \leq i < n} G[i][mid] \quad (\text{say}) = G[\underline{r}][mid]$$

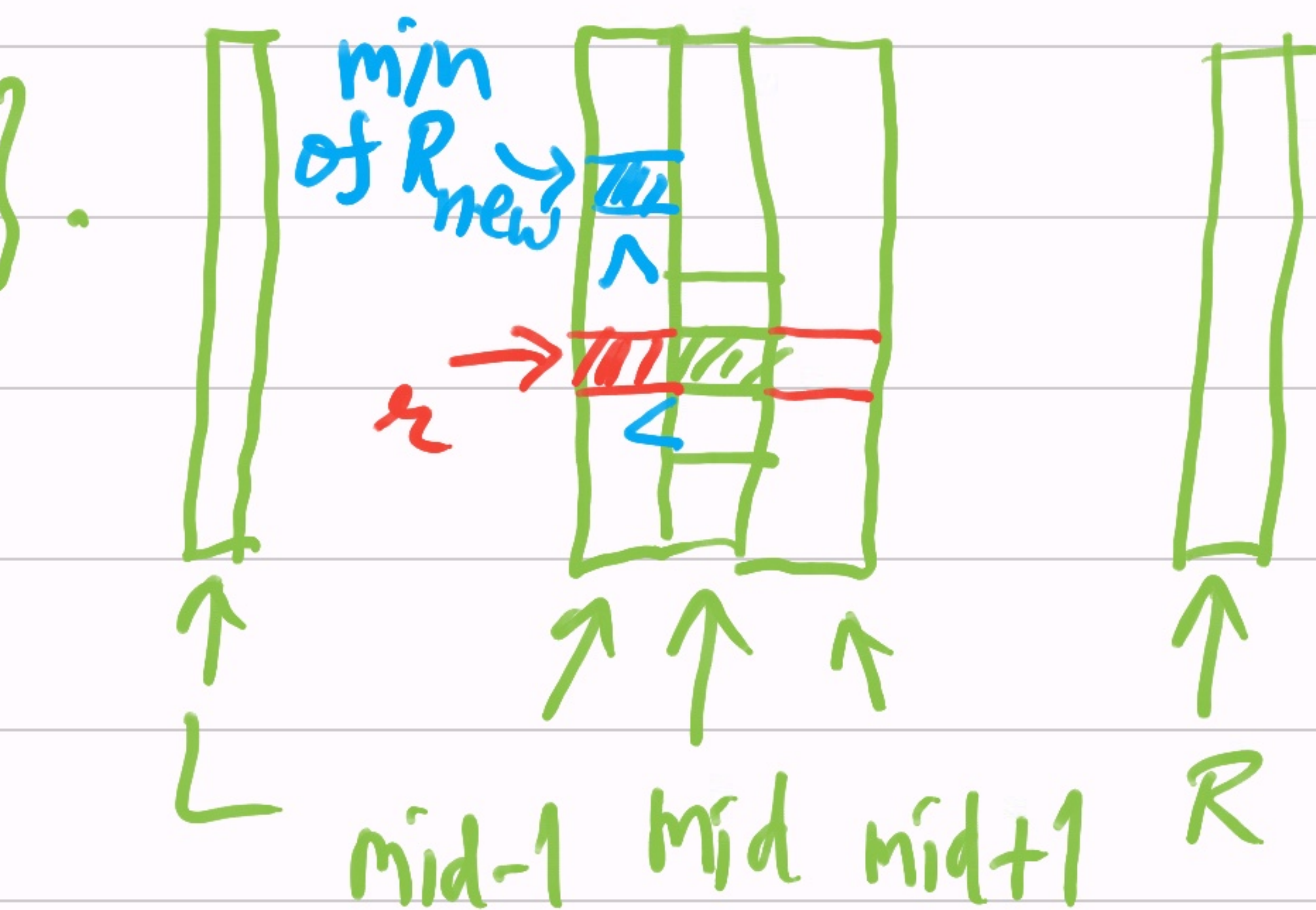
- Find $P := \min \{G[r][mid-1], G[r][mid], G[r][mid+1]\}$.

▷ If $P = G[r][mid]$ then
 OUTPUT P .

- Say, $P = G[r][mid-1]$.

Then, $R \leftarrow mid-1$.

- Search in $G[*][L \dots R]_{\text{new}}$.



next-half to search recursively.

— Formulate an invariant, similar to the 1-D-Lemma 2.

just outside L $\swarrow$ row where $\min G[*][L]$ resides

Lemma: $G[*][L-1] > G[\min_L][L]$; $G[\min_R][R] < G[*][R+1]$;

[Invariant] $\swarrow$ just outside R

This holds at every step of the recursion.

Pf: — Eg. step as done above that updated R ,
by moving to the left-half:

$$\begin{aligned} & G[\min_R][R] < G[r][R] < G[r][R+1] \\ & < G[*][R+1] \end{aligned}$$

$\Rightarrow$ 2nd bound. [1st is symmetric].

$\uparrow$
mid old

— This covers the induction-step.

— Ex: Identify the base-case.

$\Rightarrow$ The pf. is finished by induction
on #cols, $\square$

Lemma: • This algo. halves the #cols. and stops
(#steps) in $O(\lg n)$ calls.

• $G[\min_L][L]$ is a local-minima, when
 $L=R$.

Pf: • $G[*][L-1] > G[\min_L][L] < G[*][R+1]$.
& $G[\min_L-1][L] > \quad \quad \quad < G[\min_L+1][L]$.

$\Rightarrow G[\min_L][L]$ is local-min. in the
end when $L=R$.

- # recursive calls is $O(\lg n)$ as we're halving the columns. $\square$

Theorem: Local-minima in $G[n][n]$ is found in
 $O(n \lg n)$ time

Pf: — $O(\lg n)$ calls by the prev. Lemma.
— In each round, it takes $O(n)$ time
to find the min in mid-col.

$$\Rightarrow \sum_{i=1}^{\lg n} O(n) \leq O(n \lg n). \quad \square$$

— Further ideas : • Reduce $\sum_{i \leq \lg n} n \leq O(n \lg n)$

to $\sum_{0 \leq i \leq \lg n} n/2^i \leq O(n)$.

- We can try this by taking the mid-row & picking upper/lower halves!
- Alternate between halving #cols & #rows .

— How to argue, what's the invariant?

Qn: Does the previous algo. directly work?

Exercise: Find an input instance, where the col-boundary invariant, followed by the row- " " , doesn't give the right answer.

→ col-invariant can get violated when you use mid-row.

— Instead, we want a more general invariant for all the 4 borders:

Qn: $\left[\begin{array}{l} \exists ? x \text{ s.t. } x < \text{each} \\ \text{element in the border?} \end{array} \right]$



Invariant

— We need to change the algorithm only slightly to ensure the existence of λ in each recursion-step!

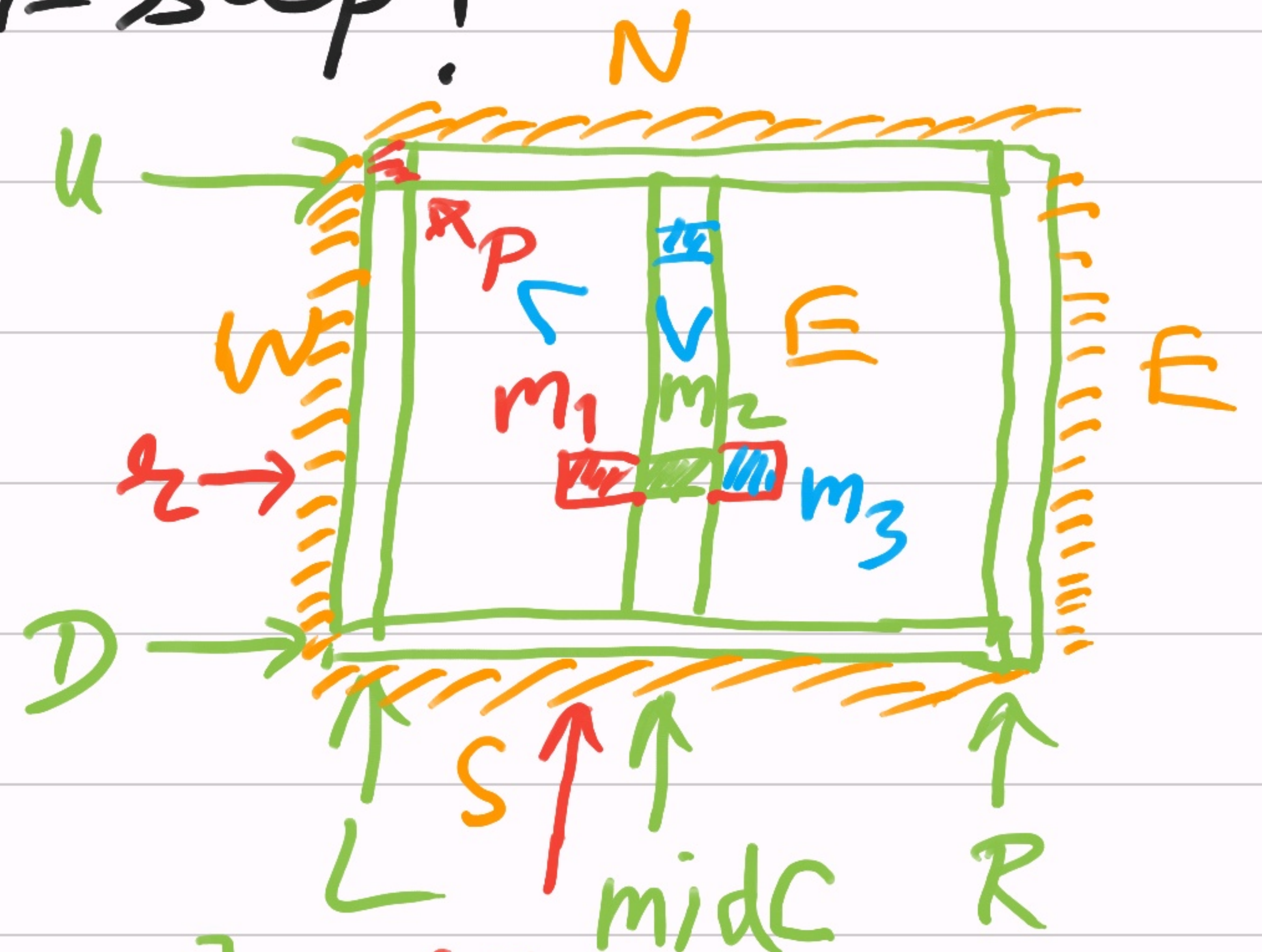
Pseudocode-fragment 1:

— Let's see the base-case first: $L = U = 0$

$$R = D = n-1$$

— Compare $\{G[r][midC-1], G[r][midC+1], P := G[u][L]\} =: S^{(midC-1)}$

— Move to the half where $(\min S)$ lies; $P \leftarrow \min S;$



-eg. If $P < m_3 < m_1, m_2$, still we pick the left-half (not the right one; unlike the old algo.)

$\Rightarrow P < \text{each cell in } \underbrace{N, S, W, E}_{\text{vacuums in the base case}} \text{ boundaries.}$
 $\nwarrow$ new pseudocode with P

$\Rightarrow$ We've covered the base-case & the immediate next-case; the induction-step is identical (for midC or midR).

$\rightarrow$ Note: P gets updated in each induction-step; so it certifies the invariant property.

Pseudocode fragment 2: Repeat the above fragment with the middle row $\text{midR} = \frac{U+D}{2}$.

Algo: Fragment-1; Fragment-2; REPEAT till 1×1 grid & OUTPUT it.

Theorem: Local-min ($G[n][n]$) runs in $O(n)$ time.

Pf: Idea: Divide-and-Conquer in 2D.

• The invariant makes sure that the final 1×1 grid is a local-minima.



- $\# \text{steps} \approx \sum_{i \leq \lg n} [\text{global-min in } \frac{n}{2^i}\text{-array}]$

$$\hookrightarrow \sum_{1 \leq i \leq \lg n} O(n/2^i) \leq O(n). \quad \square$$

Geometric-Progression

— Practice pfs. of correctness; identify the invariant; analyze the recurrence to derive the time-complexity.

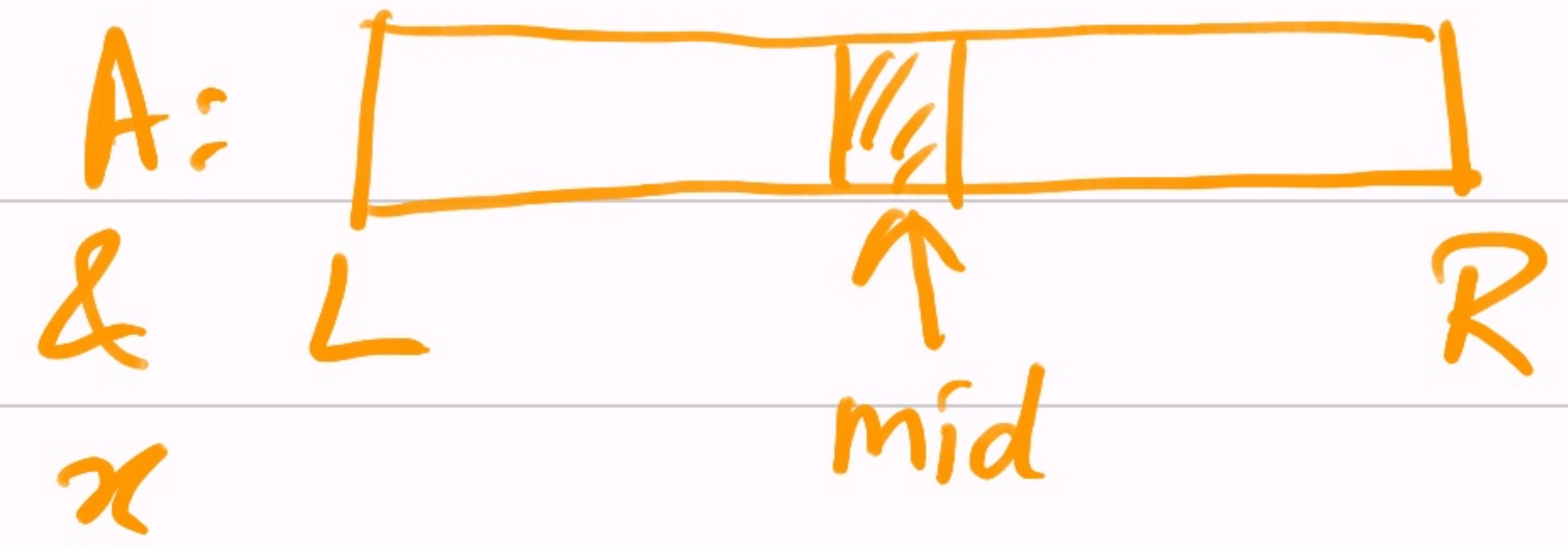
— Let's revisit some earlier exs.:

E.g. 1: Binary Search:

Invariant:

Pick the (unique) half where

x can possibly appear in A at all.



(1) $\triangleright x \in A \Leftrightarrow x$ is in one of the two halves.

(2) $\triangleright$ Which half? This is found in $O(1)$ time.

(3) $\triangleright$ The array-size halves in each call.

$\Rightarrow \#calls \leq O(\lg n).$

$\triangleright$ Binary-search time = $O(\lg n).$

- Ex. 2. Euclid's GCD algorithm. m-bits

Invariant: $\gcd(a, b) = \gcd(b, a \bmod b)$. $\gcd(a, b) \{ // a \geq b$

$$[(qb+r, b) = (r, b) = (b, r)]$$

▷ Each step in the algo. is $\leq O(m)$ [for m-bits] = linear(input).

▷ Every two iterations, b gets halved. $\Rightarrow O(\lg b) = O(m)$ iterations.

Exercise: Prove this; also, by Fibonacci recurrence.

Theorem: GCD is almost-linear time.

```
• while (b ≠ 0)
  t ← b;
  b ← a mod b;
  a ← t;
• return a;
```

Data Structure : organizes the data inside the memory (RAM/HDD) s.t.

- the space needed is compact;
- query/update is fast;
(# queries is huge)
- 1-time preprocessing can be slow, but not too slow.

Challenging Problem: Range-min in an array.
Query: $i \leq j$.
Output: $\min A[i, \dots j]$.